

Software Requirements Specification (SRS)

Project Title: Salon Management System

Core Stack: Express.js Framework & MySQL Relational Database

1. Introduction & Engineering Scope

1.1 Purpose

This specification document outlines the backend architectural design, RESTful API endpoints, database schemas, and data validation rules for the **Salon Management System** backend application. The core objective of this software system is to replace manual appointment notebooks and paper-based tracking methods with a centralized, high-performance digital platform. It serves as a comprehensive development blueprint for engineers creating secure, scalable enterprise business logic layers for modern salon chains.

1.2 Scope

The scope of this engineering delivery is focused strictly on the server-side architecture, database design, and business logic execution. The resulting application will expose a structured RESTful API layer capable of supporting various client side clients (web/mobile). The system manages 7 core business entities: customers, appointments, barbers, services, attendance logs, wage calculation structures, and analytical reports.

1.3 Core Technical Objectives

Students must implement and demonstrate mastery over the following architectural mechanics during execution:

- **Express.js API Architecture:** Robust request handling, router isolation, and scalable design execution patterns.
- **Relational Database Design:** Normalizing structural operational tables inside a MySQL/MSSQL engine with strict reference definitions.
- **Security Mechanics:** Implementing state-of-the-art JSON Web Token (JWT) stateless session tracking alongside secure password hashing layers.
- **Role-Based Authorization Middleware:** Intercepting incoming payloads to evaluate structural group authorization policies before parsing resource routes.
- **Advanced Business Automation Engines:** Writing algorithmic validation scripts preventing calendar slot collision over-allocation, ensuring payroll/commission synchronization, and rendering aggregated raw data datasets.

2. System Architecture & Directory Organization

The backend application follows a highly structured, decoupled layer approach separating network routes, input interceptors, controllers, service calculations, and data persistence operations.

2.1 Suggested Project Directory Tree

Plaintext

src/

|

- |— config/ # Database driver pool connections and environment setup keys
- |— controllers/ # HTTP Request parsing pipelines and response management blocks
- |— services/ # Central core domain business logic and algorithmic computations
- |— repositories/ # Direct MySQL data interaction queries and persistence drivers
- |— routes/ # REST Endpoint path declarations mapped to controllers
- |— middlewares/ # Authentication guards and role configuration validation scripts
- |— validations/ # Request payload validation rule sets (e.g., Joi, Zod structural forms)
- |— utils/ # Shared helper routines, formatting files, and calculation files
- |— models/ # Structural definitions of entity schema definitions
- |— docs/ # Automated API specification records (Swagger/OpenAPI YAML files)
- └─ app.js # Server entry configuration script, error handling hooks, and listener boot

3. Database Schema Blueprint (MySQL Relational Schema)

The database engine must enforce data integrity using standard relational design rules. Below are the mandated table schemas that students must configure within the MySQL database cluster:

3.1 Users Table

Contains authorization metadata and operational access flags for internal staff entries.

- id: INT, Primary Key, Auto-Increment
- name: VARCHAR(255), Not Null
- email: VARCHAR(255), Unique, Not Null
- password: VARCHAR(255), Not Null (Must store as an encrypted hash value)
- role: ENUM('Administrator', 'Receptionist', 'Barber'), Not Null
- status: ENUM('Active', 'Inactive'), Default 'Active'
- created_at: TIMESTAMP, Default CURRENT_TIMESTAMP

3.2 Customers Table

Tracks customer records independent of authentication access parameters.

- id: INT, Primary Key, Auto-Increment
- name: VARCHAR(255), Not Null
- phone: VARCHAR(20), Not Null

- email: VARCHAR(255), Nullable
- gender: ENUM('Male', 'Female', 'Other'), Nullable
- created_at: TIMESTAMP, Default CURRENT_TIMESTAMP

3.3 Services Table

Stores available treatment data catalogs along with pricing schedules and estimated runtimes.

- id: INT, Primary Key, Auto-Increment
- service_name: VARCHAR(255), Not Null
- duration: INT, Not Null (Represented in minutes)
- price: DECIMAL(10,2), Not Null
- description: TEXT, Nullable

3.4 Barbers Table

Stores specialization tags and compensation tracking criteria for the styling staff.

- id: INT, Primary Key, Auto-Increment
- user_id: INT, Foreign Key referencing Users(id) on delete restrict
- specialization: VARCHAR(255), Nullable
- commission_percentage: DECIMAL(5,2), Not Null (e.g., 40.00 representing 40%)
- joining_date: DATE, Not Null

3.5 Appointments Table

Tracks reservation status pipelines, links clients with styling resources, and registers scheduling states.

- id: INT, Primary Key, Auto-Increment
- customer_id: INT, Foreign Key referencing Customers(id)
- barber_id: INT, Foreign Key referencing Barbers(id)
- service_id: INT, Foreign Key referencing Services(id)
- appointment_date: DATETIME, Not Null
- status: ENUM('Pending', 'Confirmed', 'In Progress', 'Completed', 'Cancelled'), Default 'Pending'
- remarks: TEXT, Nullable

3.6 Attendance Table

Captures time punch tracking and historical verification logs for computing payouts.

- id: INT, Primary Key, Auto-Increment

- barber_id: INT, Foreign Key referencing Barbers(id)
- check_in: DATETIME, Not Null
- check_out: DATETIME, Nullable
- date: DATE, Not Null

3.7 Wage_Records Table

Historical database summarizing structured monthly baseline rewards plus automated commission calculations.

- id: INT, Primary Key, Auto-Increment
- barber_id: INT, Foreign Key referencing Barbers(id)
- month: VARCHAR(7), Not Null (Represented as YYYY-MM structure context)
- salary: DECIMAL(10,2), Default 0.00 (Base salary tier tracking)
- commission: DECIMAL(10,2), Default 0.00 (Aggregated commission earnings)
- total_amount: DECIMAL(10,2), Not Null (Summation validation: salary + commission)

4. Role-Based Access Control (RBAC) Permitted Matrix

The system prevents path unauthorized leaks by filtering access privileges using custom middleware blocks. The permission matrix maps specific actions to designated user roles:

Role / Operational Permission Scope	Administrator	Receptionist	Barber
Manage Barbers / Stylist Profiling	Full Access	No Access	No Access
Service Catalog Adjustments	Full Access	No Access	No Access
Customer Profiling & Log Discovery	Full Access	View Only	No Access
Appointment Lifecycle Administration	Full Access	Full Access	No Access
Time Schedule/Daily Grid Allocation	Full Access	Full Access	No Access
Attendance Review & Registration	Full Access	No Access	Mark Personal Only

Role / Operational Permission Scope	Administrator	Receptionist	Barber
Wages & Payroll Processing Modules	Full Access	No Access	View Personal
Corporate Financial Dashboard Analytics	Full Access	No Access	No Access

5. Functional Core Modules & Endpoint Directory

5.1 Module 1: Authentication Engine

- POST /api/auth/login - Validates incoming user identity records and returns structured bearer token strings.
- POST /api/auth/logout - Blacklists or clears session states for structural authorization paths.
- PUT /api/auth/change-password - Updates authorization strings following strict format updates.

5.2 Module 2: Customer Management Subsystem

- GET /api/customers - Fetches search-filtered and paginated lists of customer data entries.
- GET /api/customers/:id - Locates isolated client identity metrics along with historical visit logs.
- POST /api/customers - Creates new client data records inside the database.
- PUT /api/customers/:id - Updates biographical metrics, phone information, or details for an existing record.
- DELETE /api/customers/:id - Removes customer entity listings from active operational rows.

5.3 Module 3: Service Configuration Controls

- GET /api/services - Accesses the entire menu catalog (Haircut, Beard Styling, Hair Coloring, Facial, Head Massage).
- POST /api/services - Appends innovative new grooming packages along with cost parameters.
- PUT /api/services/:id - Adjusts active pricing models or changes treatment durations.
- DELETE /api/services/:id - Removes stale treatments from active visibility filters.

5.4 Module 4: Barber Profile Administration

- GET /api/barbers - Lists available styling staff members along with their specialization tags.
- POST /api/barbers - Appends a newly hired stylist record to the specialized team database.

- PUT /api/barbers/:id - Updates specific commission configurations or adjust shift tracking assignments.
- DELETE /api/barbers/:id - Flags employee configurations as inactive or removes them from active profiles.

5.5 Module 5: Appointment Lifecycles

- GET /api/appointments - Displays multi-criteria appointment scheduling lists matching requested dates.
- GET /api/appointments/:id - Inspects detailed lifecycle information for an individual visit entry.
- POST /api/appointments - Schedules a new visit entry (Defaults status flag parameter directly to "Pending").
- PUT /api/appointments/:id - Updates status values across standard operational pipelines: Pending, Confirmed, In Progress, Completed, or Cancelled.
- DELETE /api/appointments/:id - Erases scheduling entries or removes misplaced reservation records.

5.6 Module 6: Advanced Time Slot Scheduling Engine

- GET /api/slots - Checks active barber calendars to list open time windows.
- POST /api/slots - Configuration routes to block custom operational windows or set custom operational hours.
- **Mandated Core Business Logic Mechanics:**
 - **Double-Booking Overlap Checker:** The engine must intercept execution loops to ensure a stylist is never assigned overlapping tasks during matching timestamps.
 - **Shift Window Validation Helper:** The platform must verify that requested service start and end times fall strictly within a stylist's regular work hours.
 - **Calendar Exception Validator:** Intercepts entries to block appointments from being scheduled on official business holidays.

5.7 Module 7: Employee Attendance Trackers

- GET /api/attendance - Admin matrix review routes tracking historical check-in actions across operational centers.
- POST /api/attendance/checkin - Registers current timestamp data directly to open daily attendance cards.
- POST /api/attendance/checkout - Closes the active time card tracking window for the current date.

5.8 Module 8: Automated Wage & Commission Calculations

- GET /api/wages - Calculates payroll variables and returns summarized statements for selected staff members.
- POST /api/wages - Finalizes monthly balance tables and logs financial payout distributions.
- **Mandated Operational Commission Business Logic:**
 - Custom logic must process completed appointments automatically using individual performance shares.
 - *Calculation Framework Rule Configuration Example:* Given a base Haircut Price priced at ₹300 combined with a designated Barber Commission target tier set to 40%:
 - Stylist Commission Share: $\$300 \times 0.40 = \text{\text{₹}120}$
 - Corporate Retained Revenue Share: $\$300 - 120 = \text{\text{₹}180}$

5.9 Module 9: Corporate Dashboard Reporting Endpoints

- GET /api/reports/daily-revenue - Computes summarized operational income metrics for the current business date.
- GET /api/reports/monthly-revenue - Computes income performance distributions across active fiscal terms.
- GET /api/reports/top-services - Rank-orders standard menu options by total purchase volume to pinpoint high-margin items.
- GET /api/reports/barber-performance - Summarizes appointment tracking metrics, total hours worked, and accumulated sales commissions for each stylist.
- GET /api/reports/customer-visits - Analyzes customer return rates and tracks client retention frequencies across different branches.

6. 12-Week System Implementation Roadmap

Week 1: Requirement Analysis & Database Foundations (22/06/2026 to 28/06/2026)

- **Main Focus:** Business criteria verification, mapping relational data entity models, setting constraint rules, and planning endpoints.
- **Developer Task Assignments:** Create standardized logical ER diagrams using proper indexing keys, establish clear foreign key constraints, and map target routes.
- **Milestone Artifact:** Approved database Entity-Relationship configuration map and detailed API path manifests.

Week 2: Environment Initialization & Setup (29/06/2026 to 05/07/2026)

- **Main Focus:** Bootstrapping the application runtime framework, configuring environmental secrets files, and testing relational data connection persistence drivers.

- **Developer Task Assignments:** Configure the base configuration boilerplate using contemporary Node execution systems, set up automatic connection pool recovery layers, and design shared global error interceptor schemas.
- **Milestone Artifact:** A compiled, working base infrastructure tree capable of establishing stable query handshakes with the database.

Week 3: Authorization Guards & Session Isolation (06/07/2026 to 12/07/2026)

- **Main Focus:** Engineering onboarding access portals, generating state security signatures, and creating route access interception layers.
- **Developer Task Assignments:** Program secure hash generation routines for user passwords, implement login routers that return encrypted JWT values, and write authentication guard hooks to extract and verify identity credentials from request headers.
- **Milestone Artifact:** Secure authentication modules enforcing role-based verification checking across protected API paths.

Phase – 1 Evaluation (13/07/2026 to 17/07/2026)

Week 4: Customer Profile Data Subsystems (13/07/2026 to 19/07/2026)

- **Main Focus:** Deploying the storage registry engine and search routers for handling customer metadata profiles.
- **Developer Task Assignments:** Build standardized database interaction scripts to query customer information, write regular expression validators to scrub input fields, and implement SQL pagination logic using explicit index range delimiters.
- **Milestone Artifact:** Full CRUD operational pipelines supporting scalable customer profile lookups.

Week 5: Service Management Integration (20/07/2026 to 26/07/2026)

- **Main Focus:** Developing the configuration catalog module for the service pricing registry.
- **Developer Task Assignments:** Develop route logic to insert and modify core pricing metrics, enforce strict range checks on input data types, and write sanitation routines to process descriptive text elements safely.
- **Milestone Artifact:** A structural catalog manager handling the business service options list.

Week 6: Barber Profile Infrastructure (27/07/2026 to 02/08/2026)

- **Main Focus:** Building structural database modules to manage stylist records, specializations, and working availability rules.
- **Developer Task Assignments:** Design profiles for the salon's styling personnel, construct relational associations linking internal user identities with operational stylist profiles, and establish tracking tables for baseline performance indicators.
- **Milestone Artifact:** Operational barber management services handling skills mapping configurations.

Week 7: Appointment Management Lifecycle (03/08/2026 to 09/08/2026)

- **Main Focus:** Launching primary processing services for appointment lifecycle requests.
- **Developer Task Assignments:** Create transaction wrappers to handle new reservation inserts, write update routes to handle status phase updates, and implement relational join operations to assemble customer and stylist data payloads.
- **Milestone Artifact:** Comprehensive core appointment controllers managing system workflow pipelines.

Phase – 2 Evaluation (10/08/2026 to 14/08/2026)

Week 8: Advanced Slot Conflict Engines (10/08/2026 to 16/08/2026)

- **Main Focus:** Coding strict real-time calendar validation routines to eliminate over-scheduling anomalies.
- **Developer Task Assignments:** Program high-performance data verification queries to detect overlapping time commitments, intercept scheduling attempts that violate official calendar exceptions, and lock resource records during reservation writes to maintain absolute data accuracy.
- **Milestone Artifact:** A rock-solid scheduling validation engine that completely eliminates calendar collision events.

Week 9: Attendance Tracking Integration (17/08/2026 to 23/08/2026)

- **Main Focus:** Setting up internal tracking features to record staff punch times and summarize operational attendance logs.
- **Developer Task Assignments:** Program endpoint routers to capture daily check-in and check-out events, calculate active shifts on a daily basis, and write summary reporting scripts to help managers track absolute operational compliance.
- **Milestone Artifact:** Fully automated timecard tracking models linking shift data directly with worker identities.

Week 10: Automatic Commission & Payroll Subsystems (24/08/2026 to 30/08/2026)

- **Main Focus:** Building automatic financial calculations components to process commission tiers and compute aggregate employee compensation metrics.
- **Developer Task Assignments:** Write computational scripts that calculate commissions using fixed percentage cuts from completed service transactions, configure atomic update operations to balance ledger logs, and create summary tables to record completed payouts.
- **Milestone Artifact:** Automated commission settlement modules managing accurate payroll distribution logs.

Week 11: Enterprise Dashboard Analytical Pipelines (31/08/2026 to 13/09/2026)

- **Main Focus:** Writing high-performance mathematical calculation queries to extract strategic business intelligence and generate system performance reports.

- **Developer Task Assignments:** Write aggregate query routines to compile periodic earnings statistics, program filtering tools to rank service performance metrics, and optimize data transformation functions to format results into clean arrays for front-end charting components.
- **Milestone Artifact:** Advanced analytics controller hubs generating formatted business intelligence records.

Week 12: Rigorous Testing, Documentation & Production Launch (14/09/2026 to 20/09/2026)

- **Main Focus:** Resolving technical bugs, documenting the API surface with interactive tools, optimizing database queries, and launching the application to public staging servers.
- **Developer Task Assignments:** Run comprehensive automated testing suites to audit all API pathways, create interactive OpenAPI definitions using integrated tools, fix any slow-running queries by adding appropriate database indexes, and deploy the live application package to remote hosting environments.
- **Milestone Artifact:** A production-deployed live backend instance accompanied by comprehensive structural Swagger interface documentation[cite: 4].

Phase – 3 Evaluation (21/09/2026 to 03/10/2026)

7. Evaluation Rubric Matrix

Student design portfolios will be strictly evaluated based on the following 100-mark distribution matrix:

Assessment Component Criterion	Weightage Maximum	Focus Verification Area
REST API Structural Standards	15 Marks	Proper usage of semantic HTTP verbs, accurate application of standard response codes, clean URL structure paths, and proper integration of query parameters[cite: 4].
Database Normalization & Performance	15 Marks	Proper layout of relational constraints, implementation of appropriate indexing strategies to optimize speed, data type allocation, and transactional reliability[cite: 4].
JWT Access Management Security	15 Marks	Implementation of secure password hashing standards, reliable validation of token lifetimes, and robust role-based middleware access controls[cite: 4].

Assessment Component Criterion	Weightage Maximum	Focus Verification Area
Algorithmic Scheduling Controls	15 Marks	Complete prevention of booking conflicts, robust handling of concurrent schedule requests, and valid implementation of operational calendar rules[cite: 4].
Automated Payroll Computations	15 Marks	Error-free execution of financial mathematics functions, accurate tracking of variable commission configurations, and secure ledger transaction operations[cite: 4].
Analytical Query Performance	10 Marks	Efficient execution of complex data aggregations and fast performance when summarizing metrics across large, multi-table datasets[cite: 4].
System Exception Management	5 Marks	Implementation of comprehensive error-catching loops across all application layers, preventing server crashes and returning clean error messages[cite: 4].
OpenAPI Documentation Quality	5 Marks	Delivery of a clear and complete Swagger API playground that details all available parameter fields, schemas, and example response payloads[cite: 4].
Production Server Deployments	5 Marks	Successful cloud server initialization featuring secure environment variables and stable connectivity with remote database instances[cite: 4].
TOTAL SCORING CAPACITY	100 Marks	<i>Note: Extra credit rewards may apply if advanced testing scripts are implemented alongside baseline criteria.</i>

This finishes the comprehensive university-grade Backend SRS and Architectural Blueprint for the **Salon Management System**. Submit your tracking documentation whenever you are ready to begin project review loops!